

Approfondimento

Versioning: 11 casi d'uso

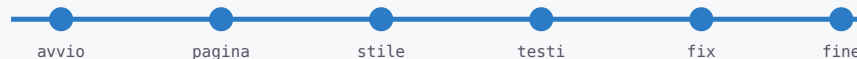
gli stessi concetti (commit, branch, merge, release) in situazioni tutte diverse

Ogni caso ha un **grafo** (pallino = commit/versione, linea = tempo che scorre da sinistra a destra) e una **spiegazione di massima**. Serve per imparare a **leggere** la storia di un progetto. Informatica.

Come si legge un grafo: ogni **pallino** è un **commit** (una fotografia salvata). La **linea orizzontale** in alto è di solito **main**, il ramo principale. Le linee colorate che si staccano sono **branch** (rami di lavoro). Quando due linee si riuniscono è un **merge**. Le **bandierine gialle** sono le **release** (versioni pubblicate, es. v1.0).

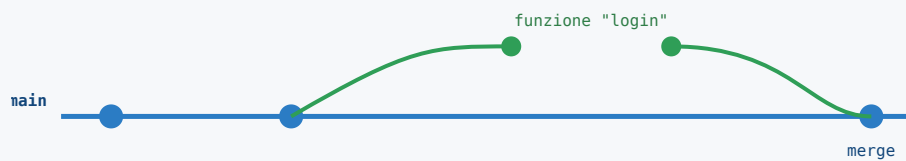
1. Sviluppo lineare (da soli, tutto dritto)

Il caso più semplice: un solo autore, nessun ramo. Ogni commit segue il precedente in fila. La storia è una linea dritta. Va benissimo per progetti piccoli.



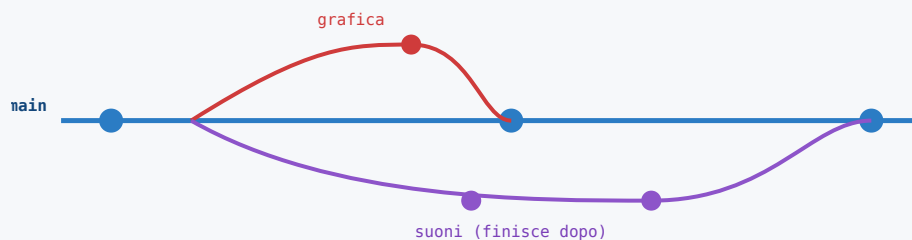
2. Una funzione su un branch, poi merge

Provi una funzione nuova su un ramo separato, così il main resta sempre funzionante. Quando è pronta, la **unisci** (merge) al main.



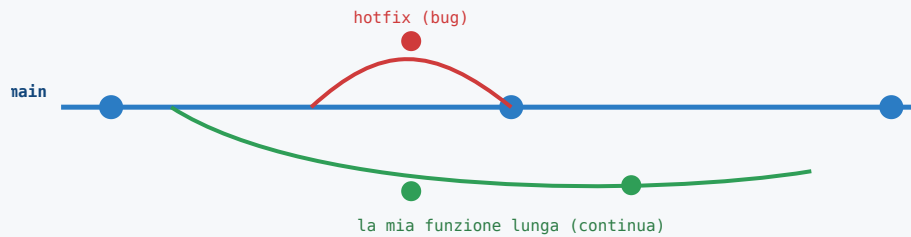
3. Due funzioni in parallelo, merge in tempi diversi

Due rami partono insieme ma finiscono in momenti diversi: uno si fonde prima, l'altro dopo. Il main raccoglie i pezzi man mano che sono pronti.



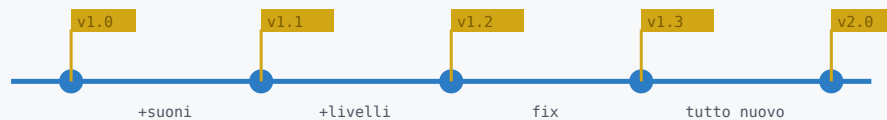
4. Hotfix urgente durante una funzione lunga

Stai lavorando a una funzione che richiede giorni, ma esce un **bug grave** in produzione. Apri un ramo **hotfix** corto, lo correggi e lo unisci subito; poi torni alla tua funzione.



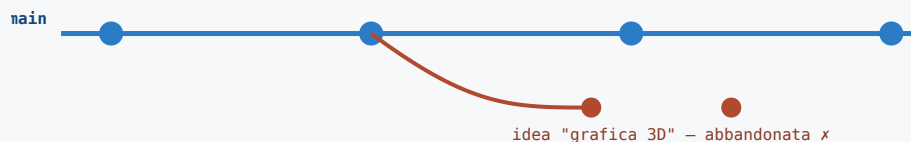
5. Le release pubblicate (bandierine sul main)

Man mano che il progetto avanza, si **pubblicano** le versioni: `v1.0` giocabile, poi correzioni e aggiunte fino a un cambiamento grosso `v2.0`. Ogni bandierina è una versione "congelata".



6. Un ramo abbandonato (idea scartata)

Non tutte le idee funzionano. Qui un ramo viene provato ma **mai unito**: si decide che non va e lo si lascia lì. Il bello di Git: il main non ne risente per niente.



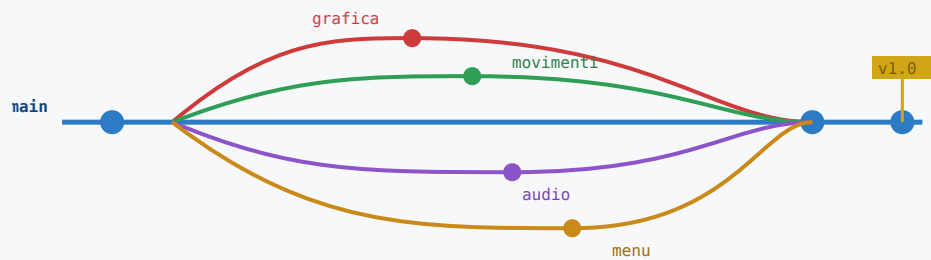
7. Un ramo che nasce da un altro ramo

A volte una funzione è così grande che, mentre la sviluppi, ne stacchi un pezzo su un **sotto-ramo**. Prima rientra il sotto-ramo nel ramo, poi il ramo nel main. Rami dentro rami.



8. Grande integrazione (progetto di gruppo)

Nel lavoro di squadra ognuno ha il suo ramo (grafica, movimenti, audio, menu). Alla fine confluiscono tutti nel main con le loro Pull Request: nasce la versione completa.



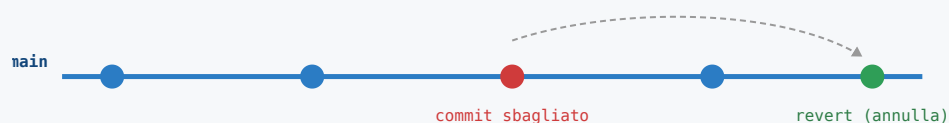
9. Ramo di manutenzione a lungo termine

Esci con la `v1.0` e il `main` va avanti verso la `v2.0` (tutto nuovo). Ma chi usa ancora la `v1` ha bisogno di correzioni: si tiene un **ramo di manutenzione** parallelo dove escono `v1.1`, `v1.2` solo con i fix.



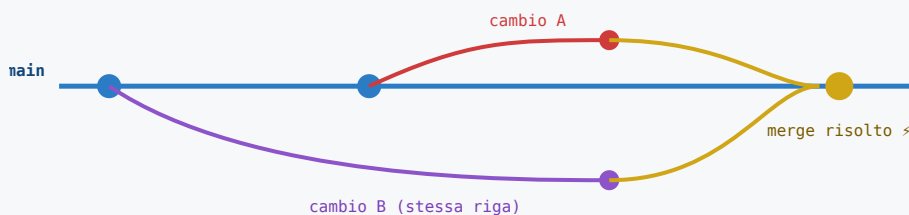
10. Revert: annullare una modifica sbagliata

Un commit ha introdotto un problema. Invece di cancellare la storia (non si fa), si aggiunge un **nuovo commit** che **annulla** quello sbagliato: è il **revert**. La storia resta onesta e il progetto torna a posto.



11. Conflitto di merge risolto

Due rami hanno cambiato **la stessa riga** in modo diverso. Quando si uniscono, Git non sa quale tenere: è un **conflitto**. Una persona decide cosa tenere e chiude il merge con un commit speciale di "risoluzione".



In sintesi: con pochi elementi — commit, branch, merge, tag/release — si descrivono **tutte** le situazioni di un progetto reale, dalla più semplice (linea dritta) alle più complesse (integrazione di squadra, manutenzione parallela, conflitti). Saper **leggere** questi grafi è già metà del lavoro.